
Hallucination is Inevitable: An Innate Limitation of Large Language Models

Ziwei Xu Sanjay Jain Mohan Kankanhalli
School of Computing, National University of Singapore
ziwei.xu@u.nus.edu {sanjay,mohan}@comp.nus.edu.sg

Abstract

Hallucination has been widely recognized to be a significant drawback for large language models (LLMs). There have been many works that attempt to reduce the extent of hallucination. These efforts have mostly been empirical so far, which cannot answer the fundamental question whether it can be completely eliminated. In this paper, we formalize the problem and show that it is impossible to eliminate hallucination in LLMs. Specifically, we define a formal world where hallucination is defined as inconsistencies between a computable LLM and a computable ground truth function. **By employing results from learning theory, we show that LLMs cannot learn all the computable functions and will therefore inevitably hallucinate if used as general problem solvers.** Since the formal world is a part of the real world which is much more complicated, hallucinations are also inevitable for real world LLMs. Furthermore, for real world LLMs constrained by provable time complexity, we describe the hallucination-prone tasks and empirically validate our claims. Finally, using the formal world framework, we discuss the possible mechanisms and efficacies of existing hallucination mitigators as well as the practical implications on the safe deployment of LLMs.

1 Introduction

The emergence of large language models (LLMs) has marked a significant milestone in the field of artificial intelligence, particularly in natural language processing. These models, with their vast knowledge bases and ability to generate coherent and contextually relevant text, have greatly impacted research, industry, and society. **However, one of the critical challenges they face is the problem of "hallucination," where the models generate plausible but factually incorrect or nonsensical information.** This issue has brought increasing concerns about safety and ethics as LLMs are being applied widely, resulting in a growing body of literature trying to classify, understand, and mitigate it.

Prior works have identified multiple possible sources of hallucination in LLMs from the data collection to the training and inference aspects. For example, in the survey paper [32], the authors attribute hallucination in natural language generation to heuristic data collection, innate divergence, imperfect representation learning, erroneous decoding, exposure bias, and parametric knowledge bias. A plethora of methods have been proposed to mitigate hallucination. For example, factual-centred metrics [21, 22, 47, 63] and benchmarks [39, 40, 72] have been proposed to measure and reduce hallucination on specific datasets. Retrieval-based methods reinforce LLM by knowledge graphs or databases to help correct factual errors [63, 81]. Prompting the models to reason [76] and verify [15] their answers has also been shown to reduce hallucination.

Up to now, research on LLM hallucination remains largely empirical. Useful as they are, empirical studies cannot answer the fundamental question: *can hallucination be completely eliminated?* The answer to this question is fundamental as it indicates a possible upper limit of LLMs' abilities. How-

ever, since it is impossible to empirically enumerate and test every possible input, formal discussion on this question is impossible without a clear definition and formal analysis of hallucination.

In the real world, formally defining hallucination, a factual or logical error of LLM, turns out to be extremely difficult. This is because a formal definition of semantics in the real world is still an open problem [14, 64]. Hence in this work, we rigorously define a formal world of computable functions, wherein precise discussions on hallucination is feasible. In this world, hallucination occurs whenever an LLM fails to exactly reproduce the output of a computable function. Under this definition, we present a fundamental result that *hallucination is inevitable for any computable LLM, regardless of model architecture, learning algorithms, prompting techniques, or training data*. Since this formal world is a part of the real world, the result also applies to LLMs in the real world. To the best of our knowledge, this is the first work that formally defines hallucination and discusses its inevitability. A parallel work by Kalai and Vempala [33] provided a statistical lower bound on the rate of hallucination for calibrated LLMs. It shows a significant fact that pretraining LLMs for predictive accuracy leads to hallucination even with perfect training data. The results in this paper is comparably more general as they are applicable to all computable LLMs.

The contributions of this paper are: (1) we formally define and discuss hallucination for LLMs and, by employing results in learning theory [3, 4, 20, 31], we show that hallucination is inevitable for all computable LLMs; (2) we discuss practical implications of theoretical results on the hallucination mitigators and deployment of LLMs in the real world; (3) we show examples of hallucination-prone problems and validate the claims by empirical study.

2 Definitions

In this section, we detail our definitions of LLMs, hallucination, and training and deployment of LLMs under the hallucination context. Readers are referred to Appendix A for a table of notation.

2.1 Large Language Model

We start with the definitions for our alphabet and strings:

Definition 1 (Alphabet and Strings). Let $\mathbb{N}$ be the set of natural numbers. An alphabet $\mathcal{A}$ is a finite set of N tokens $\mathcal{A} = \{a_0, a_1, \dots, a_{N-1}\}$. A string is a sequence $w_{0:n-1} = w_0 w_1 \dots w_{n-1}$ obtained by concatenating tokens for n times, where $i, n, N \in \mathbb{N}, w_i \in \mathcal{A}$. Let $\mathcal{S}$ be a computable set¹ of all the finite-length strings of alphabet $\mathcal{A}$ and $(s_0, s_1, \dots)$ be an one-to-one enumeration of all the elements in $\mathcal{S}$.

For real-world LLMs, tokens are usually unique combinations of alphanumerical characters. An LLM is a probabilistic model of a string that conditions the output at time t based on all the tokens that come before it in the string. The most common usage of LLM is to complete a partial string $w_{0:q-1}$, or a prompt, into a complete string $w_{0:n-1}$, where $n > q$, by maximizing the likelihood of the latter in a token-by-token manner. Readers are referred to Appendix B for implementation details of LLMs.

To facilitate more general discussions, we now ignore all the implementation details of an LLM and only focus on its input and output behavior after training. Though the current LLMs are resource constrained, in this paper we consider trained LLMs to be total computable functions (usually, as functions from $\mathcal{S}$ to $\mathcal{S}$).

An LLM is trained to adapt to training samples before being deployed (a general training procedure is detailed in Procedure 1 in Section 2.3). This means an LLM has different states depending on the training samples it has seen in a training process². We use $h^{[i]}$ to denote the state of LLM h at the i^{th} stage of training (for example, after the LLM is trained on i samples $\{(s_0, f(s_0)), (s_1, f(s_1)), \dots, (s_{i-1}, f(s_{i-1}))\}$, where f is a ground truth function. We assume that the training process as well as all the states of LLMs after training are uniformly computable (that is,

¹A computable set [68] is a set whose membership is decidable by a computable function.

²In practice, most LLMs are discussed in an offline setting, where it is trained on a certain set of finite data and then deployed without further training. This is a special case of the setting in this paper, where we consider hallucination for different states of a given LLM based on different sets of training samples.

given an LLM and the training samples, we can effectively obtain the program of an LLM state after it is trained on certain part of the training samples at some stage).

All real-world LLMs have some properties, for example, they all complete their computation in polynomial time. In our discussion below, we will also consider computably enumerable sets of LLMs where all the member LLMs have a particular property P . Following the definition, a set of P -property LLMs is a proper subset of all LLMs. This classification is useful for our further discussion about LLMs' limitations, where we will view different LLMs with different properties as different subsets of total computable functions.

Different from general total computable functions, LLMs can be categorized by the desirability of its outputs along a spectrum. On the "nonsensical" side is a clueless token predictor which produces meaningless outputs given input strings s . On the "ideal" side, a hallucination-free function produces sensible and factual outputs for any well-formed input strings. In between are the real-world LLMs: their outputs are comprehensible most of the time; however, they occasionally "hallucinate" and generate non-factual statements. This spectrum and the relation between concepts in this section is shown in Fig. 1.

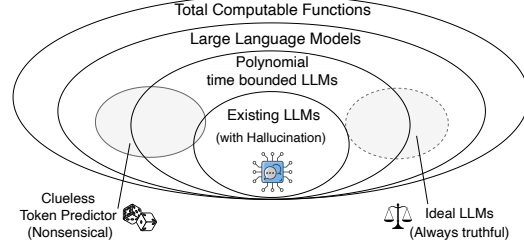


Figure 1: Relations between LLMs, P -property LLM given some P (e.g., P is polynomial time bounded), and computable functions. The set of ideal LLMs is not attainable, as will be shown.

2.2 A Formal World and Hallucination

Hallucination is in essence an erroneous output produced by an LLM. Without getting tangled in the difficult problem of formalizing "correctness" in our real world, we define hallucination in a formal world where f is the ideal function that produces correct output (e.g., completion) $f(s)$ for any input string (or prompt, question etc.) $s \in \mathcal{S}$.

Definition 2 (Formal World of f). A formal world of ground truth function f is a set $\mathcal{G}_f = \{(s, f(s)) \mid s \in \mathcal{S}\}$, where $f(s)$ is the only correct output of input string s , for all $s \in \mathcal{S}$.

The training samples $\mathcal{T}$ is defined as a set of input-output pairs we get from the formal world of f :

Definition 3 (Training Samples $\mathcal{T}$). Training samples $\mathcal{T}$ is a set $\{(s_0, y_0), (s_1, y_1), \dots, (s_i, y_i), \dots \mid y_i = f(s_i), s_i \in \mathcal{S}, i \in \mathbb{N}\}$.

Set $\mathcal{T}$ is a generalized corpus of what f outputs given input strings. For example, if f answers "true" for factual inputs and "false" otherwise, then $\mathcal{T}$ could look like $\{("A shark is a mammal.", "false"), ("Earth orbits around the Sun.", "true"), \dots\}$. On the other hand, if f is a function that completes or answers input string s , then $\mathcal{T}$ could look like $\{("Is shark a fish or mammal?", "Fish."), ("What is the sum of binary numbers 10 and 11?", "101."), \dots\}$.

With the ground truth f introduced, it is straightforward to define hallucination in the formal world as when all the states of an LLM fails to fully reproduce the output of ground truth function f . Formally:

Definition 4 (Hallucination). An LLM h is hallucinating with respect to a ground truth function f , if $\forall i \in \mathbb{N}, \exists s \in \mathcal{S}$ such that $h^{[i]}(s) \neq f(s)$.

2.3 Training an LLM

In the formal world, using our definition of hallucination in Definition 4, the core question of whether hallucination can be eliminated can be translated to:

Question 1 (The Fundamental Question). Can an LLM h be trained following a fixed procedure, such that for any ground-truth function f , $\exists i \in \mathbb{N}, \forall s \in \mathcal{S}, h^{[i]}(s) = f(s)$?

We are not interested in the training and deployment details like model initialization, selection of optimizers, learning rates, objective functions, stopping criteria, inference hyperparameters (e.g., temperature,) and so on, and wish our discussion to be independent of these factors. Therefore, we

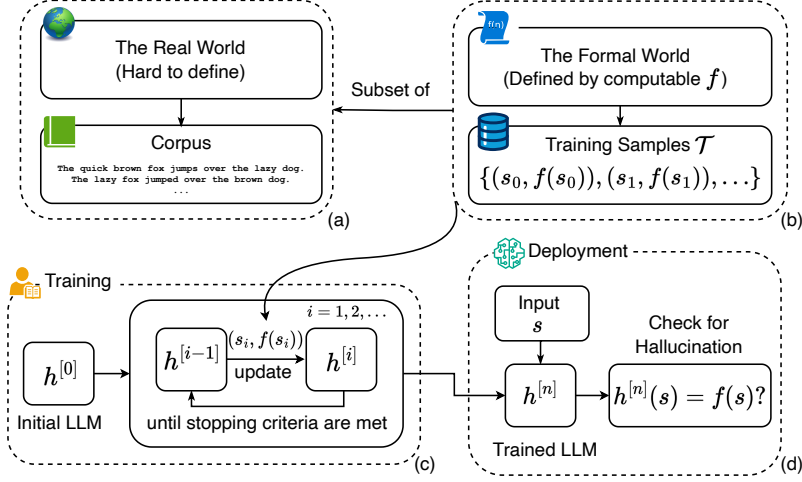


Figure 2: Illustration of relations between all concepts defined in Section 2. (a) shows the real-world corpus, which is a superset of (b) the formal world of ground truth function f and training samples $\mathcal{T}$. (c) shows the procedure that trains LLM h as defined in Procedure 1, which takes training samples and updates h until the stopping criteria are met. Finally in (d), the LLM at a state n is deployed and produces output given an unseen string s . Hallucination is defined by comparing LLM’s answer $h^{[n]}(s)$ with ground truth value $f(s)$.

lump together all the factors and call it a “training and deploying” procedure. In this procedure, an LLM h is iteratively updated by training samples. We check the LLM for hallucination during deployment, after the stopping criteria are met and the training procedure stops. Formally:

Procedure 1 (Training and deploying an LLM).

- Input: A *stream*^a of training samples $\mathcal{T} = ((s_0, f(s_0)), (s_1, f(s_1)), \dots)$.
- Output: $h^{[i]}$, an LLM in a trained state indexed by i , where $i \in \mathbb{N}$.
- Procedure:
 1. Let $i = 0$. Let $h^{[0]}$ be the LLM in the initial state (e.g., with randomly initialized parameters).
 2. Training and Validation Iteration:
 - (a) If stopping criteria are met (LLM is ready): end iteration, goto step 3.
 - (b) Retrieve a training sample $(s_i, f(s_i))$ from $\mathcal{T}$.
 - (c) Update LLM $h^{[i]}$ to $h^{[i+1]}$ according to training samples $\{(s_j, f(s_j)) \mid j \leq i\}$.
 - (d) Let $i \leftarrow i + 1$, go to step 2a.
 3. Deployment: Let $h^{[i]}$ be the final trained state. End the procedure.

^aAssuming no limitations on their numbers.

There are a few notes about this procedure:

- In step 2a, traditional stopping criteria include a preset number of iterations or stagnating loss functions. The procedure above does not assume any criterion, meaning the training can take arbitrarily many finite samples and arbitrarily long finite time.
- As a result of this procedure, $h^{[i]}$ is a state of LLM h in which it is trained on samples $\{(s_j, f(s_j)) \mid j < i\}$. In particular, $h^{[0]}$ is an LLM state in which h is initialized but never trained.
- The trained LLM $h^{[i]}$ can be expected to: (1) answer $f(s) = ?$, or (2) given “ s ,” complete it with “ $f(s)$ ”, or (3) given “is it true that $f(s) = t$?” answer “yes” or “no”. The three cases are equivalent if (a) for every input the correct output is unique, and (b) the time limit for LLM

is unbounded, which is the case in our discussion. For ease of presentation and without loss of generality, we assume case (1), where LLM uses $h^{[i]}(s)$ to answer the question in the rest of the paper.

An illustration showing the relations between all the definitions in this section is shown in Fig. 2. It is noteworthy that LLMs trained by Procedure 1 are far more powerful and flexible than their counterparts in the real world. Therefore, *if hallucination is inevitable for our LLMs in the relatively simple formal world, then hallucination is inevitable for LLMs in the more complicated real world.*

3 Hallucination is Inevitable for LLMs

In this section, we show that hallucination is inevitable in the formal world by suitably applying results in learning theory literature. Specifically, we will use the diagonalization argument in our proof. The diagonalization argument was originally used to prove that some infinite sets (e.g., the set of real numbers) are larger than others (e.g., the set of natural numbers) by Cantor [9, 17]. The high-level idea is to show that any enumeration of an uncountable set is not exhaustive by constructing a counter example that is in the set but not in the enumeration [77]. The argument gets its name because such counter example is constructed by flipping the diagonal entries, i.e., the i^{th} component of the i^{th} element, in the table showing the enumeration.

This section is organized in a restricted-to-general manner, where we first discuss less complex LLMs resembling real-world ones and then generalize to any computable LLMs. *First*, we show that all states of all the LLMs in any computably enumerable sets will hallucinate on some inputs (Theorem 1). *Then*, we show that all states of all the LLMs in a computably enumerable set will hallucinate on infinite many inputs (Theorem 2). *Finally*, we show that hallucination is inevitable for all computable LLMs (Theorem 3). At the end, we will answer the fundamental Question 1.

3.1 Computably Enumerable LLMs will Hallucinate

As all the currently proposed LLMs are polynomial time bounded, these LLMs (considered as functions) are contained in a computably enumerable set of LLMs. To prove that this set of LLMs must hallucinate, below we prove a stronger theorem that *any* computably enumerable set of LLMs (including this set) will inevitably hallucinate, using the classic diagonalization argument.

Theorem 1. For all computably enumerable sets of LLMs $\{h_0, h_1, \dots\}$, there exists a computable ground truth function f , such that all $h_i^{[j]}$, $i, j \in \mathbb{N}$, will hallucinate.

Table 1: Illustration of diagonalization for the proof of Theorem 1. Each row represents a particular LLM state. Each column represents a string s_i with which the LLM is tested. Blue cells are the outputs used to construct f in Eq. (1)

LLMs	Test Samples				
	s_0	s_1	s_2	s_3	$\dots$
$\hat{h}_0$	$\hat{h}_0(s_0)$	$\hat{h}_0(s_1)$	$\hat{h}_0(s_2)$	$\hat{h}_0(s_3)$	$\dots$
$\hat{h}_1$	$\hat{h}_1(s_0)$	$\hat{h}_1(s_1)$	$\hat{h}_1(s_2)$	$\hat{h}_1(s_3)$	$\dots$
$\hat{h}_2$	$\hat{h}_2(s_0)$	$\hat{h}_2(s_1)$	$\hat{h}_2(s_2)$	$\hat{h}_2(s_3)$	$\dots$
$\hat{h}_3$	$\hat{h}_3(s_0)$	$\hat{h}_3(s_1)$	$\hat{h}_3(s_2)$	$\hat{h}_3(s_3)$	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$
f	$\Delta(\hat{h}_0(s_0))$	$\Delta(\hat{h}_1(s_1))$	$\Delta(\hat{h}_2(s_2))$	$\Delta(\hat{h}_3(s_3))$	$\dots$

Proof. Consider any computably enumerable set of LLMs $\{h_0, h_1, \dots\}$. An LLM h_i can be trained on no training sample, one training sample, two training samples, and so on, resulting in its different states $\{h_i^{[0]}, h_i^{[1]}, \dots, h_i^{[j]}, \dots\}$, $i, j \in \mathbb{N}$. To facilitate diagonalization argument, we use the Cantor pairing function [8] to re-enumerate all these LLM states as $\{\hat{h}_0, \hat{h}_1, \dots, \hat{h}_k, \dots\}$, where $h_i^{[j]}$ is re-enumerated as $\hat{h}_k$ with $k = (i + j)(i + j + 1)/2 + j$, $\forall i, j \in \mathbb{N}$. In order to demonstrate the contradiction, let us assume that at least one of the LLM states is hallucination-free (if there is none, the theorem is trivially proved). Feeding a string s to LLMs in all the enumerated states, we get

their outputs as $\{\hat{h}_0(s), \hat{h}_1(s), \dots\}$. Repeating this for all $s \in \mathcal{S}$, we get a table as in Table 1, which contains all the outputs of all the states of all the LLMs in the set being considered.

Now we define the ground truth function f so that $f(s_i)$ contradicts the elements along the diagonal of the table (see blue cells in Table 1):

$$f(s_i) = \Delta(\hat{h}_i(s_i)), \forall i \in \mathbb{N}, \quad (1)$$

where Δ is a computable function that returns a different string from its input. For example, we could define $\Delta(s_k) = s_{k+1}$, which returns the next string of s_k in $\mathcal{S}$. Therefore $\Delta(\hat{h}_i(s_i)) \neq \hat{h}_i(s_i)$. By the above construction and according to Definition 4, $\hat{h}_0$ hallucinates w.r.t. f as $f(s_0) \neq \hat{h}_0(s_0)$ and so for $\hat{h}_1$ as $f(s_1) \neq \hat{h}_1(s_1)$. Repeating this reasoning for all $s_i \in \mathcal{S}$, we see that all states of all the LLMs in the given set will hallucinate w.r.t. f because $f(s_i) \neq \hat{h}_i(s_i)$ for all $i \in \mathbb{N}$. This contradicts with our assumption and thus proves the theorem. $\square$

LLMs will Hallucinate on Infinitely Many Questions One might argue that Theorem 1 only shows that all states of all the LLMs in any computably enumerable set will hallucinate on one but not all inputs. Therefore, hallucination in the formal world could be negligible. However, we can show the following theorem:

Theorem 2. For all computably enumerable sets of LLMs $\{h_0, h_1, \dots\}$, there exists a computable ground truth function f , such that all $h_i^{[j]}$, $i, j \in \mathbb{N}$, hallucinate on infinitely many inputs.

Proof. Consider the following computably enumerable set of LLMs: $\{h_0, h_1, \dots\}$. Similar to the proof of Theorem 1, we enumerate all the states of these LLMs as $\{\hat{h}_k | k \in \mathbb{N}\}$ using the Cantor pairing function. Feeding a string s to these LLMs, we get their answers as $\{\hat{h}_0(s), \hat{h}_1(s), \dots\}$.

Define a computable function f as follows:

$$f(s_i) = \Delta(\{\hat{h}_j(s_i) \mid j \leq i\}), \forall i \in \mathbb{N},$$

where $\Delta(\{\hat{h}_j(s_i) \mid j \leq i\})$ returns a string that is different from all the strings in set $\{\hat{h}_j(s_i) \mid j \leq i\}$.

By the above construction, $\forall i \geq j, f(s_i) \neq \hat{h}_j(s_i)$ because the value of $f(s_i)$ is explicitly defined as a string different from $\hat{h}_j(s_i), i \geq j$ (see Table 2). Therefore, $\hat{h}_0$ will hallucinate on $s_0, s_1, \dots$; and $\hat{h}_1$ will hallucinate on $s_1, s_2, \dots$. In general, LLM $\hat{h}_k$ will hallucinate on all input strings after s_{k-1} in the one-to-one enumeration $(s_0, s_1, \dots)$ of $\mathcal{S}$. $\square$

Table 2: Illustration of proof for Theorem 2. Each row represents a particular LLM. Each column represents a string s_i where the LLM is tested on. A value in the j^{th} column of i^{th} row represents $\hat{h}_i(s_j)$. Blue cells are the outputs that we use to construct f , on which the LLM in the corresponding row will hallucinate.

LLMs	Test Samples				
	s_0	s_1	s_2	s_3	$\dots$
$\hat{h}_0$	$\hat{h}_0(s_0)$	$\hat{h}_0(s_1)$	$\hat{h}_0(s_2)$	$\hat{h}_0(s_3)$	$\dots$
$\hat{h}_1$	$\hat{h}_1(s_0)$	$\hat{h}_1(s_1)$	$\hat{h}_1(s_2)$	$\hat{h}_1(s_3)$	$\dots$
$\hat{h}_2$	$\hat{h}_2(s_0)$	$\hat{h}_2(s_1)$	$\hat{h}_2(s_2)$	$\hat{h}_2(s_3)$	$\dots$
$\hat{h}_3$	$\hat{h}_3(s_0)$	$\hat{h}_3(s_1)$	$\hat{h}_3(s_2)$	$\hat{h}_3(s_3)$	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$
f	$\Delta(\{\hat{h}_0(s_0)\})$	$\Delta(\{\hat{h}_j(s_1) \mid j \leq 1\})$	$\Delta(\{\hat{h}_j(s_2) \mid j \leq 2\})$	$\Delta(\{\hat{h}_j(s_3) \mid j \leq 3\})$	$\dots$

3.2 Any Computable LLM will Hallucinate

In this section, we go one more step towards a direct answer to the fundamental question by considering LLMs as general total computable functions. This step can be achieved by using Theorem 1 and 2. Any *individual* computable LLM h forms a set $\{h\}$, which only contains h itself and is thus computably enumerable. Therefore, applying Theorem 1 on this set, we know that there

exists a ground truth function f such that all states of this particular h will hallucinate. Furthermore, applying Theorem 2, we know that some f can make hallucination happen for infinitely many inputs. This can be done for any total computable LLM, hence the following theorem similar to [20]:

Theorem 3. For all computable LLMs h , there exists a computable ground truth function f such that each of $h^{[j]}, j \in \mathbb{N}$ hallucinates w.r.t. f . Furthermore, there exists a computable ground truth function³ f' such that each of $h^{[j]}, j \in \mathbb{N}$ hallucinates on infinitely many inputs w.r.t. f' .

As a specific example, by adapting results from [20], we present a special case where f represents a computable linear order, which is an abstraction of many real-world relations such as ranking of movies or songs, chronological ordering of events, alphabetical sorting of words, and so on.

Theorem 4. For all computable LLM h , there exists a computable ordering $<$ such that LLM h hallucinates when answering question “ $s_{2n+1} < s_{2n}?$ ” after being trained on training samples $\{(“s_i < s_j?” , f_{<}(s_i < s_j?)) \mid i, j < 2n\}$, where $n \in \mathbb{N}$, $f_{<}(s_i < s_j?) = “yes”$ if $s_i < s_j$ and “no” otherwise.

Appendix D provides proof, analysis, and empirical study for this theorem.

An obvious but important corollary of Theorem 3 is that it is impossible to use an LLM to eliminate its hallucination:

Corollary 1. All computable LLMs cannot prevent themselves from hallucinating.

If there exists an LLM h' which can prevent itself from hallucination, then for any computable function f , there exists a state of h' which is hallucination-free w.r.t. f , which contradicts Theorem 3. It explicitly suggests that methods relying on LLMs themselves to mitigate hallucination, such as prompt-based chain of thoughts [76], cannot *eliminate* hallucination.

Answer to the Fundamental Question It has been shown that hallucination is inevitable for (1) LLMs in any computably enumerable set (Theorem 1 and 2), and (2) any total computable LLMs (Theorem 3). Since the formal world is a part of the real world, and real-world LLMs are a part of total computable LLMs, we conclude with a negative answer to the fundamental question: *hallucination of LLMs cannot be eliminated in the real world*. This is independent of implementation details such as (1) LLMs’ architecture, (2) training procedure, (3) input prompts and questions, and so on, as long as they follow the definitions in Section 2 (which all real-world LLMs follow).

4 Discussion

4.1 Identifying Hallucination-Prone Problems

The idea behind all the theorems in Section 3 is that, for given (sets of) LLMs, we find computable ground truth function f that all of the LLMs cannot learn, and therefore they will inevitably hallucinate therein. Therefore, one way to identify hallucination-prone problems is to identify what cannot be computed by the given set of LLMs. Table 3 provides an incomplete list of such problems. In each problem listed in the table, ground truth f is expected to have the following behavior:

- Combinatorial List: f lists all the strings with length n using an alphabet of two characters. Computing f takes $\Omega(2^n)$ time.
- Presburger arithmetic [56, 65]: Presburger arithmetic is the first-order theory of natural numbers with addition and order $<$. Given a statement in the arithmetic, f returns “yes” if the statement can be proved within the arithmetic and “no” otherwise. Computing f takes $\Omega(2^{2^{cn}})$ time, for some $c > 0$ [18].
- Subset Sum: given a set of n integers and a number q , f returns “yes” when there is a subset that sums up to q and “no” otherwise. This problem is NP-complete.
- Boolean Satisfiability (SAT) [13]: given a formula of n Boolean variables, f returns “yes” if there exists an assignment on these variables which results in the formula to be true and “no” otherwise. This problem is NP-complete [13].

³In this theorem f and f' differs for different h , while f in Theorem 1 and 2 is constructed for all LLMs in the set being considered.

Table 3: Identified hallucination-prone problems for different sets of LLMs. $O(n^k)$ and $O(2^n)$ indicate polynomial-time and exponential-time complexity, respectively.

LLMs	Problem	Hallucination Guaranteed by	Extra Assumption
$O(n^k)$ time bounded LLMs (e.g., all existing LLMs)	Combinatorial List	Requiring $\Omega(2^n)$ solution	None
	Subset Sum	Being NP-complete	$P \neq NP$
	Boolean Satisfiability (SAT)	Being NP-complete [13]	$P \neq NP$
	Entailment of Propositional Logic	Being co-NP-complete [2]	$P \neq NP$
$O(2^n)$ time bounded LLMs, $O(n^k)$ time bounded LLMs	Presburger Arithmetic [56, 65]	Requiring $\Omega(2^{2^{2^n}})$ solution [18]	None
All Computable LLMs (incl. all LLMs above)	Learning all Computable Linear Orders	Theorem 4	None
	Solving all Computable Problems	Theorem 3	None
	Entailment of First-Order Logic	Being Undecidable [68]	None

- Entailment of propositional logic: let $M(\psi)$ and $M(\phi)$ be the set of all the models of propositional logic formula ψ and ϕ respectively. f returns “yes” if $M(\psi) \subseteq M(\phi)$ and “no” otherwise. This problem is co-NP-complete [2].
- Entailment of first-order logic: let $M(\psi)$ and $M(\phi)$ be the set of all the models of first-order logic formula ψ and ϕ respectively. f returns “yes” if $M(\psi) \subseteq M(\phi)$ and “no” otherwise. This problem is undecidable [68].

As a result, real-world LLMs’ answers about mathematical problems and logic reasoning should always be subject to proper scrutiny. The result is supported by a recent study [25]. In Appendix C, we provide an empirical study to illustrate the combinatorial list problem. In Appendix D, we detail the problem of learning all computable linear orders and provide empirical results.

Furthermore, problems equivalent to those listed in Table 3 are also hallucination-prone. As a result, LLMs’ answers about mathematical and logic reasoning should be subject to proper scrutiny.

4.2 Existing and Possible Hallucination Mitigators

We discuss efficacies and limitations of existing and possible hallucination mitigators as follows.

Larger models, model ensembles, and more training data LLMs are believed to show “emergent abilities” [75] because of the significantly increased number of parameters and training samples. Therefore, it is natural to think that hallucination will disappear as LLMs grow larger. In the context of this work, increasing model parameters boosts the complexity of LLMs, enabling it to capture more complex ground truth functions. Increasing the size of $\mathcal{T}$ helps rule out invalid LLM candidates and helps with convergence during training [70]. However, increasing parameters and data is futile if the ground truth function f cannot be captured by LLMs at all as suggested in Section 3. For example, adding multi-head attention layers to a polynomial-time LLM will result in a larger polynomial-time LLM and will only reduce and possibly eliminate hallucination w.r.t. polynomial-time ground truth functions. It will not eliminate hallucination w.r.t. an exponential-time ground truth function, no matter how many layers or training data are added. Similarly, a model ensemble is essentially a single LLM. It is bounded by Theorem 3 and cannot eliminate hallucination.

Prompting LLMs with Chain of Thoughts/Reflections/Verification This approach belongs to the larger family of in-context learning, which provides example solutions or relevant knowledge about a target task in the prompt [15, 69, 76, 79]. Essentially, its intuition is that f generates answer in a procedural manner. Complex problems often have multiple solutions with varying degrees of complexity. Prompting is effective in mitigating hallucination by guiding them towards solutions more preferred by humans, which are possibly the ones with lower complexities, and within LLMs’ capabilities. For example, a recursive approach to compute Fibonacci sequence requires exponential time, whereas dynamic programming solves the same problem in linear time. However, it is unlikely that all ground-truth functions f can be fully described by prompts. Therefore, this approach will

only work for specific tasks. Moreover, as suggested by Corollary 1, it is impossible to eliminate hallucination by changing the prompts and hope LLM can automatically eliminate its hallucination.

Guardrails and Fences The guardrails are principles that align LLMs’ output with human values, ethics, and legal requirements. The fences is a list of critical tasks that should never be fully automated using LLMs. Both of them serve as safety constraints to prevent LLMs (and other AI models) from generating undesirable outcomes. Guardrails and fences can be formally programmed [59] to explicitly affect LLMs’ behaviours. Therefore, it is more powerful than training samples defined in Definition 3 and is potentially a useful hallucination mitigator for the formal world and some real-world problems. However, its scalability in the real world remains an open problem.

Knowledge-Enhanced LLMs This approach uses external knowledge (e.g., knowledge graphs and databases) and symbolic reasoning (e.g., logics) to aid LLMs both in training and inference. Popular chatbots driven by LLMs, such as ChatGPT, has started utilizing tools such as search engine, code interpreter, and calculators to solve complex problems beyond their innate capabilities [51]. Similar to programmable guardrails and fences, it explicitly controls the LLM workflow by changing how information is recalled through retrieval from knowledge database [11, 37, 45, 55]. In this way, LLMs receive extra information about the ground truth function f other than via training samples. Therefore, Theorem 3 is inapplicable herein. This is potentially an effective mitigator of hallucination in the formal world. However, its scalability in real-world tasks is an open problem.

4.3 Practical Implications

We believe the discussions so far have the following implications for AI practitioners.

All LLMs trained only with input-output pairs will hallucinate when used as general problem solvers. This is guaranteed by Theorem 3. Problems beyond LLMs’ computation capabilities are hallucination-prone. Note that a problem could be intellectually simple (for human) but computationally hard (for LLMs), and vice versa. So it is not immediately obvious whether a problem is (not) hallucination-prone for LLMs. Furthermore, computation complexity is only one of the many reasons for hallucination in the real world. For example, imperfect training data can lead to hallucination even in computationally easy tasks.

Another side note is that this implication only applies to “useful” LLMs, in the sense that the LLM must eventually give an answer for a question outside of its training data. Specifically, it can refuse to answer an arbitrary number of questions (e.g., by saying “I don’t know”) before it answers one. Whenever an LLM (in a state) answers a question outside of the training data it has seen, diagonalization technique in proofs for Theorem 1-2 can be applied to that answer. If an LLM never answers, then it will never hallucinate. On the other hand, as long as it answers for some unseen questions, it will hallucinate in some formal world.

Without external aids like guardrails, fences, knowledge base, and human control, LLMs cannot be used automatically in any safety-critical decision-making. This is a direct corollary of the conclusion above. External aids could help LLMs overcome limitations stated in Section 3 as it provides information beyond training samples of input-output pairs. Furthermore, humans’ role is paramount because human values are arguably more complex than validity. Making safety-critical decisions usually requires rational and humane (e.g., understanding, empathy, and ethicality) judgements that are otherwise very difficult, if not impossible, to compute. Any errors due to hallucination in these cases, for example, making decisions about human life, are very likely to be unacceptable.

Research and regulations on LLMs’ safety boundaries is crucial and urgent in ensuring sustainable development of LLMs. Real-world financial loss has been reported, where LLMs used for customer service provides incorrect information [5]. Hallucination in automated sensing and actuation, such as in a robotics setting, might result in dangerous real-world outcomes. It is therefore important for theorists and practitioners to reach consensus on the ability boundaries for LLMs. There should also be appropriate regulations to prevent LLM uses outside of their ability boundaries. A preliminary attempt towards an “upper bound” of LLMs’ capabilities is presented in Appendix E.

4.4 Limitations

While this work has answered a fundamental question about hallucination, we note its limitations. *First*, it does not account for hallucinations on problems within LLMs’ computational capabilities. We believe the reasons behind hallucination is multifaceted; formal discussion is one facet that we feel is under-emphasized and under-explored. *Second*, it assumes deterministic ground truth function and thus provides little insights from the probabilistic perspectives. *Third*, in empirical study, existing LLMs are used without being further finetuned. Nevertheless, we believe these limitations are not critical to the question answered in the paper, but constitute open problems in Appendix H.

5 Related Works

This section provides a concise overview of pertinent studies in hallucination in LLMs. Recent surveys [26, 32, 43, 53, 73] provide extensive reviews of the field. We also discuss briefly the relation between this paper, PAC learnability, and online learnability in Appendix G.

5.1 Classification of Hallucination

A conventional classification of hallucination is the intrinsic-extrinsic dichotomy [16, 27, 32, 80]. Intrinsic hallucination occurs when LLM outputs contradict with the provided input, such as prompts. On the other hand, extrinsic hallucination occurs when LLM outputs cannot be verified by the information in the input. Huang *et al.* [26] extends this dichotomy by introducing faithfulness hallucination. Rawte *et al.* [58] divided hallucination into “factual mirage” and “silver lining”, denoting erroneous outputs based on factually correct or incorrect inputs.

5.2 Causes of Hallucination

Hallucination has been generally attributed to issues in data, training, and inference stages as identified in existing surveys [26, 32]. Issues in the data include poor quality [35], misinformation [40], bias [48, 54], and being outdated [38, 50] knowledge. Moreover, a fair portion of knowledge in the wild is long-tailed, making it challenging to recall during deployment [34, 44]. During training, the model could suffer from architectural and strategic deficiencies which hinder proper learning such as exposure bias [6] and diluted attention [12, 24, 42]. During inference, hallucination can also be caused by sampling randomness [1, 16], insufficient context attention [61], and softmax bottleneck [10, 78].

5.3 Mitigating Hallucination

The mitigation of hallucination involves tackling its underlying causes. For data-related issues, solutions include fact-focused datasets [19, 23] and developing automatic data cleaning techniques [49, 57, 60]. Retrieval augmentation, which uses relevant external documents to ground LLMs, can help reduce knowledge gap and reduce hallucination [63, 81]. Prompting techniques like Chain-of-Thought [76] and Tree-of-Thought [79] have been applied to improve knowledge recall and reasoning [74]. To mitigate training-related hallucination, architectural improvements and training objectives like sharpening softmax functions [42] and factuality-enhanced training objectives [36, 62] have been proposed. To overcome inference-related hallucination, new decoding methods are introduced to improve factuality or faithfulness of LLMs. Factual-nucleus sampling, proposed by Lee *et al.* [36], aims to balance the diversity and the factuality in model outputs. Chain-of-Verification, introduced by Dhuliawala *et al.* [15], prompts the LLM to self-correct their mistakes during generation.

6 Conclusion

In this paper, we study the fundamental problem of eliminating hallucinations in LLMs. To do so, we define a formal world where hallucination in LLMs can be clearly defined and discussed. Specifically, hallucination is defined as inconsistencies between computable LLMs and a computable ground truth function. By utilizing results in learning theory, we show that hallucination is inevitable for computable LLMs if the ground truth function is any computable function. Since the formal world is a part of the real world, we further conclude that it is impossible to eliminate hallucination in the

real world LLMs. We discuss the possible mechanisms and effectiveness of existing hallucination mitigators and discuss practical implications that our theoretical results have on the deployment of LLMs in the real world. We emphasize that since hallucination is inevitable, rigorous study of the safety of LLMs is critical and urgent.

References

- [1] Renat Aksitov, Chung-Ching Chang, David Reitter, Siamak Shakeri, and Yunhsuan Sung. “Characterizing Attribution and Fluency Tradeoffs for Retrieval-Augmented Large Language Models”. In: *arXiv preprint 2302.05578* (2023).
- [2] Sanjeev Arora and Boaz Barak. *Computational Complexity - A Modern Approach*. Cambridge University Press, 2009.
- [3] J. Bärzdiņš and R. Freivald. “On the prediction of General Recursive Functions”. In: *Soviet Mathematics Doklady, (Dokl. Akad. Nauk SSSR)* 13 (1972), pp. 1224–1228.
- [4] Nikolay Bazhenov, Ekaterina B. Fokina, and Luca San Mauro. “Learning families of algebraic structures from informant”. In: *Information and Computation* 275 (2020), p. 104590.
- [5] BBC. *Airline held liable for its chatbot giving passenger bad advice - what this means for travellers*. Accessed: 2024-02-22. 2024. URL: <https://web.archive.org/web/20240224015400/https://www.bbc.com/travel/article/20240222-air-canada-chatbot-misinformation-what-travellers-should-know>.
- [6] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. “Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks”. In: *Advances in Neural Information Processing Systems*. Vol. 28. 2015.
- [7] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. “Language Models are Few-Shot Learners”. In: *arXiv preprint 2005.14165* (2020).
- [8] Georg Cantor. “Ein beitrage zur mannigfaltigkeitslehre”. In: *Journal für die reine und angewandte Mathematik (Crelles Journal)* 1878.84 (1878), pp. 242–258.
- [9] Georg Cantor. “Ueber eine elementare Frage der Mannigfaltigkeitslehre.” In: *Jahresbericht der Deutschen Mathematiker-Vereinigung* 1 (1890/91), pp. 72–78.
- [10] Haw-Shiuan Chang and Andrew McCallum. “Softmax Bottleneck Makes Language Models Unable to Represent Multi-mode Word Distributions”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. May 2022, pp. 8048–8073.
- [11] Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. “Benchmarking Large Language Models in Retrieval-Augmented Generation”. In: *AAAI*. AAAI Press, 2024, pp. 17754–17762.
- [12] David Chiang and Peter Cholak. “Overcoming a Theoretical Limitation of Self-Attention”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. May 2022, pp. 7654–7664.
- [13] Stephen A. Cook. “The Complexity of Theorem-Proving Procedures”. In: *Proceedings of the Third Annual ACM Symposium on Theory of Computing*. 1971, pp. 151–158.
- [14] Kees van Deemter. “The Pitfalls of Defining Hallucination”. In: *arXiv preprint 2401.07897* (2024).
- [15] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. “Chain-of-Verification Reduces Hallucination in Large Language Models”. In: *arXiv preprint 2309.11495* (2023).
- [16] Nouha Dziri, Andrea Madotto, Osmar Zaiane, and Avishek Joey Bose. “Neural Path Hunter: Reducing Hallucination in Dialogue Systems via Path Grounding”. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Nov. 2021, pp. 2197–2214.

- [17] William Ewald. *From Kant to Hilbert: A Source Book in the Foundations of Mathematics*. Oxford University Press, Apr. 2005. ISBN: 9780198505358.
- [18] Michael J Fischer and Michael O Rabin. “Super-exponential complexity of Presburger arithmetic”. In: 1974.
- [19] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. “The Pile: An 800GB Dataset of Diverse Text for Language Modeling”. In: *arXiv preprint 2101.00027* (2020).
- [20] E. Mark Gold. “Language Identification in the Limit”. In: *Information and Control* 10.5 (1967), pp. 447–474.
- [21] Ben Goodrich, Vinay Rao, Peter J. Liu, and Mohammad Saleh. “Assessing The Factual Accuracy of Generated Text”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019, pp. 166–175.
- [22] Nuno M. Guerreiro, Elena Voita, and André Martins. “Looking for a Needle in a Haystack: A Comprehensive Study of Hallucinations in Neural Machine Translation”. In: *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. May 2023, pp. 1059–1075.
- [23] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Harkirat Singh Behl, Xin Wang, Sébastien Bubeck, Ronen Eldan, Adam Tauman Kalai, Yin Tat Lee, and Yuanzhi Li. “Textbooks Are All You Need”. In: *arXiv preprint 2306.11644* (2023).
- [24] Michael Hahn. “Theoretical Limitations of Self-Attention in Neural Sequence Models”. In: *Transactions of the Association for Computational Linguistics* 8 (2020), pp. 156–171.
- [25] Pengfei Hong, Deepanway Ghosal, Navonil Majumder, Somak Aditya, Rada Mihalcea, and Soujanya Poria. “Stuck in the Quicksand of Numeracy, Far from AGI Summit: Evaluating LLMs’ Mathematical Competency through Ontology-guided Perturbations”. In: *arXiv 2401.09395* (2024).
- [26] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions”. In: *arXiv preprint 2311.05232* (2023).
- [27] Yichong Huang, Xiachong Feng, Xiaocheng Feng, and Bing Qin. “The Factual Inconsistency Problem in Abstractive Text Summarization: A Survey”. In: *arXiv preprint 2104.14839* (2023).
- [28] HuggingFace. *Llama 2 is here - get it on Hugging Face*. Accessed: 2023-12-07. URL: <https://web.archive.org/web/20231207183448/https://huggingface.co/blog/llama2>.
- [29] HuggingFace. *Llama-2-70b Chat Model on HuggingFace*. Accessed: 2023-12-15. URL: <https://web.archive.org/web/20231215014453/https://huggingface.co/meta-llama/Llama-2-70b-chat-hf>.
- [30] HuggingFace. *Llama-3-70b-Instruct Model on HuggingFace*. Accessed: 2023-12-15. URL: <https://web.archive.org/web/20240518083325/https://huggingface.co/unsloth/llama-3-70b-Instruct-bnb-4bit>.
- [31] Sanjay Jain, Daniel N. Osherson, James S. Royer, and Arun Sharma. *Systems That Learn: An Introduction to Learning Theory*. The MIT Press, Feb. 1999. ISBN: 9780262276252.
- [32] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. “Survey of Hallucination in Natural Language Generation”. In: *ACM Computing Surveys* 55.12 (Mar. 2023).
- [33] Adam Tauman Kalai and Santosh S. Vempala. “Calibrated Language Models Must Hallucinate”. In: *Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC)*. 2024.
- [34] Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel. “Large Language Models Struggle to Learn Long-Tail Knowledge”. In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. July 2023, pp. 15696–15707.

- [35] Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. “Deduplicating Training Data Makes Language Models Better”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. May 2022, pp. 8424–8445.
- [36] Nayeon Lee, Wei Ping, Peng Xu, Mostofa Patwary, Pascale N Fung, Mohammad Shoeybi, and Bryan Catanzaro. “Factuality Enhanced Language Models for Open-Ended Text Generation”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 34586–34599.
- [37] Patrick S. H. Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Proceedings of the 34th International Conference on Neural Information Processing Systems*. 2020.
- [38] Daliang Li, Ankit Singh Rawat, Manzil Zaheer, Xin Wang, Michal Lukasik, Andreas Veit, Felix Yu, and Sanjiv Kumar. “Large Language Models with Controllable Working Memory”. In: *Findings of the Association for Computational Linguistics: ACL 2023*. July 2023, pp. 1774–1793.
- [39] Junyi Li, Xiaoxue Cheng, Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. “HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 6449–6464.
- [40] Stephanie Lin, Jacob Hilton, and Owain Evans. “TruthfulQA: Measuring How Models Mimic Human Falsehoods”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. May 2022, pp. 3214–3252.
- [41] Nick Littlestone. “Learning Quickly When Irrelevant Attributes Abound: A New Linear-threshold Algorithm”. In: *Mach. Learn.* 2.4 (1987), pp. 285–318.
- [42] Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. “Exposing Attention Glitches with Flip-Flop Language Modeling”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023.
- [43] Junliang Luo, Tianyu Li, Di Wu, Michael Jenkin, Steve Liu, and Gregory Dudek. “Hallucination Detection and Hallucination Mitigation: An Investigation”. In: *arXiv preprint 2401.08358* (2024).
- [44] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. “When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. July 2023, pp. 9802–9822.
- [45] Ariana Martino, Michael Iannelli, and Coleen Truong. “Knowledge Injection to Counter Large Language Model (LLM) Hallucination”. In: *The Semantic Web: ESWC 2023 Satellite Events*. 2023, pp. 182–185.
- [46] Meta Platforms, Inc. *Introducing Meta Llama 3: The most capable openly available LLM to date*. Accessed: 2024-04-30. URL: <https://web.archive.org/web/20231207183448/https://huggingface.co/blog/llama2>.
- [47] Anshuman Mishra, Dhruvesh Patel, Aparna Vijayakumar, Xiang Lorraine Li, Pavan Kapanipathi, and Kartik Talamadupula. “Looking Beyond Sentence-Level Natural Language Inference for Question Answering and Text Summarization”. In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. June 2021, pp. 1322–1336.
- [48] Pranav Narayanan Venkit, Sanjana Gautam, Ruchi Panchanadikar, Ting-Hao Huang, and Shomir Wilson. “Nationality Bias in Text Generation”. In: *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. May 2023, pp. 116–122.
- [49] Feng Nie, Jin-Ge Yao, Jinpeng Wang, Rong Pan, and Chin-Yew Lin. “A Simple Recipe towards Reducing Hallucination in Neural Surface Realisation”. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. July 2019, pp. 2673–2679.

- [50] Yasumasa Onoe, Michael J. Q. Zhang, Eunsol Choi, and Greg Durrett. “Entity Cloze By Date: What LMs Know About Unseen Entities”. In: *NAACL-HLT (Findings)*. Association for Computational Linguistics, 2022, pp. 693–702.
- [51] OpenAI. “ChatGPT Release Notes”. In: (2023). Accessed: 2023-12-16. URL: <https://web.archive.org/web/20231214021113/https://help.openai.com/en/articles/6825453-chatgpt-release-notes>.
- [52] OpenAI. “GPT-4 Technical Report”. In: *arXiv preprint* 2303.08774 (2023).
- [53] Liangming Pan, Michael Saxon, Wenda Xu, Deepak Nathani, Xinyi Wang, and William Yang Wang. “Automatically Correcting Large Language Models: Surveying the landscape of diverse self-correction strategies”. In: *arXiv preprint* 2308.03188 (2023).
- [54] Amandalynne Paullada, Inioluwa Deborah Raji, Emily M. Bender, Emily Denton, and Alex Hanna. “Data and its (dis)contents: A survey of dataset development and use in machine learning research”. In: *Patterns* 2.11 (2021), p. 100336.
- [55] Baolin Peng, Michel Galley, Pengcheng He, Hao Cheng, Yujia Xie, Yu Hu, Qiuyuan Huang, Lars Liden, Zhou Yu, Weizhu Chen, and Jianfeng Gao. “Check Your Facts and Try Again: Improving Large Language Models with External Knowledge and Automated Feedback”. In: *arXiv preprint* 2302.12813 (2023).
- [56] Mojzesz Presburger. “Über die vollständigkeit eines gewissen systems der arithmetik ganzer zahlen, in welchen die addition als einzige operation hervortritt”. In: *Comptes-Rendus du ler Congres des Mathematiciens des Pays Slavs*. 1929.
- [57] Vikas Raunak, Arul Menezes, and Marcin Junczys-Dowmunt. “The Curious Case of Hallucinations in Neural Machine Translation”. In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. June 2021, pp. 1172–1183.
- [58] Vipula Rawte, Swagata Chakraborty, Agnibh Pathak, Anubhav Sarkar, S. M Towhidul Islam Tonmoy, Aman Chadha, Amit P. Sheth, and Amitava Das. “The Troubling Emergence of Hallucination in Large Language Models – An Extensive Definition, Quantification, and Prescriptive Remediations”. In: *arXiv preprint* 2310.04988 (2023).
- [59] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. “NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Dec. 2023, pp. 431–445.
- [60] Lei Shen, Haolan Zhan, Xin Shen, Hongshen Chen, Xiaofang Zhao, and Xiaodan Zhu. “Identifying Untrustworthy Samples: Data Filtering for Open-Domain Dialogues with Bayesian Optimization”. In: *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 2021, pp. 1598–1608.
- [61] Weijia Shi, Xiaochuang Han, Mike Lewis, Yulia Tsvetkov, Luke Zettlemoyer, and Scott Wen-tau Yih. “Trusting Your Evidence: Hallucinate Less with Context-aware Decoding”. In: *arXiv preprint* 2305.14739 (2023).
- [62] Weijia Shi, Sewon Min, Maria Lomeli, Chunting Zhou, Margaret Li, Rich James, Xi Victoria Lin, Noah A. Smith, Luke Zettlemoyer, Scott Yih, and Mike Lewis. “In-Context Pretraining: Language Modeling Beyond Document Boundaries”. In: *arXiv preprint* 2310.10638 (2023).
- [63] Kurt Shuster, Spencer Poff, Moya Chen, Douwe Kiela, and Jason Weston. “Retrieval Augmentation Reduces Hallucination in Conversation”. In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. Nov. 2021, pp. 3784–3803.
- [64] Jeff Speaks. “Theories of Meaning”. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Edward N. Zalta. Spring 2021. Metaphysics Research Lab, Stanford University, 2021. URL: <https://plato.stanford.edu/archives/spr2021/entries/meaning/>.
- [65] Ryan Stansifer. *Presburger’s article on integer arithmetic: Remarks and translation*. Tech. rep. Cornell University, 1984. URL: <https://dl.acm.org/doi/book/10.5555/867696>.
- [66] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. “LLaMA: Open and Efficient Foundation Language Models”. In: *arXiv preprint* 2302.13971 (2023).

- [67] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. “Llama 2: Open Foundation and Fine-Tuned Chat Models”. In: *arXiv preprint* 2307.09288 (2023).
- [68] A. M. Turing. “On Computable Numbers, with an Application to the Entscheidungsproblem”. In: *Proceedings of the London Mathematical Society* s2-42.1 (1937), pp. 230–265.
- [69] Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. “Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting”. In: *NeurIPS*. 2023.
- [70] L. G. Valiant. “A Theory of the Learnable”. In: *Communications of the ACM* 27.11 (Nov. 1984), pp. 1134–1142.
- [71] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. “Attention is All You Need”. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. 2017, pp. 6000–6010.
- [72] Tu Vu, Mohit Iyyer, Xuezhi Wang, Noah Constant, Jerry Wei, Jason Wei, Chris Tar, Yun-Hsuan Sung, Denny Zhou, Quoc Le, and Thang Luong. “FreshLLMs: Refreshing Large Language Models with Search Engine Augmentation”. In: *arXiv preprint* 2310.03214 (2023).
- [73] Cunxiang Wang, Xiaozhe Liu, Yuanhao Yue, Xiangru Tang, Tianhang Zhang, Cheng Jiayang, Yunzhi Yao, Wenyang Gao, Xuming Hu, Zehan Qi, Yidong Wang, Linyi Yang, Jindong Wang, Xing Xie, Zheng Zhang, and Yue Zhang. “Survey on Factuality in Large Language Models: Knowledge, Retrieval and Domain-Specificity”. In: *arXiv preprint* 2310.07521 (2023).
- [74] Peifeng Wang, Zhengyang Wang, Zheng Li, Yifan Gao, Bing Yin, and Xiang Ren. “SCOTT: Self-Consistent Chain-of-Thought Distillation”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. July 2023, pp. 5546–5558.
- [75] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. “Emergent Abilities of Large Language Models”. In: *Transactions on Machine Learning Research* (2022).
- [76] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter brian, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 24824–24837.
- [77] Wikipedia contributors. “Cantor’s diagonal argument — Wikipedia, The Free Encyclopedia”. In: (2023). [Online; accessed 6-December-2023]. URL: https://en.wikipedia.org/w/index.php?title=Cantor%27s_diagonal_argument&oldid=1173962712.
- [78] Zhilin Yang, Zihang Dai, Ruslan Salakhutdinov, and William W. Cohen. “Breaking the Softmax Bottleneck: A High-Rank RNN Language Model”. In: *International Conference on Learning Representations*. 2018.
- [79] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. “Tree of Thoughts: Deliberate Problem Solving with Large Language Models”. In: *arXiv preprint* 2305.10601 (2023).
- [80] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lema Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, Longyue Wang, Anh Tuan Luu, Wei Bi, Freda Shi, and Shuming Shi. “Siren’s Song in the AI Ocean: A Survey on Hallucination in Large Language Models”. In: *arXiv preprint* 2309.01219 (2023).

- [81] Ruochen Zhao, Xingxuan Li, Shafiq Joty, Chengwei Qin, and Lidong Bing. “Verify-and-Edit: A Knowledge-Enhanced Chain-of-Thought Framework”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, July 2023, pp. 5823–5840.

Appendix

A Notation and Terms

Table A1: Table of Notation and Terms.

Functions	
f	A ground truth function.
h	An LLM.
$h^{[n]}$	A state of LLM h , where it is trained on the $0^{th}, 1^{st}, \dots, (n-1)^{th}$ training samples.
$\{h_0, h_1, \dots\}$	A computably enumerable set of LLMs.
$\hat{h}_k$	The k^{th} entry in an enumeration of all the states of LLMs in the set $\{h_0, h_1, \dots\}$, where for $k = (i+n)(i+n+1)/2 + n$, $\hat{h}_k$ is the same LLM as $h_i^{[n]}$, $i, n \in \mathbb{N}$.
Data	
$\mathcal{A}$	The finite alphabet of tokens.
$w_{0:n-1}$	Alternative notation of string $w_0 w_1 \dots w_{n-1}$, where $w_i \in \mathcal{A}, i \in \mathbb{N}$.
$\mathcal{S}$	The set of all finite-length strings made from tokens in $\mathcal{A}$.
s_i	The i^{th} string of an one-to-one computable enumeration $(s_0, s_1, \dots)$ of $\mathcal{S}$.
$\mathcal{G}_f$	The set $\{(s_i, f(s_i)) \mid i \in \mathbb{N}\}$ of all input-output pairs of function f on $\mathcal{S}$.
$\mathcal{T}_n$	The set of training samples $\{(s_i, f(s_i)) \mid i = 0, 1, \dots, n-1\}$.
Other Notations	
$p(w_{0:n-1})$	Probability of string $w_{0:n-1}$.
$\Delta(s)$	A string different from s .
$\Delta(\{s_0, s_1, \dots\})$	A string different from all the strings in the set $\{s_0, s_1, \dots\}$.
$L(m, \mathcal{A})$	A task that requests all the strings of length m made of tokens in alphabet $\mathcal{A}$.
$\omega(m)$	A task that asks if a statement about a linear order is true, after providing examples of m elements in that linear order.
$R(m, n)$	A task that asks for the n^{th} character of the input string with a length of m characters.
Terms	
Total Computable Function on $\mathcal{S}$	A function that produces an output in finite steps for each input in $\mathcal{S}$.

B Implementation of Large Language Models

LLMs are essentially probabilistic models of strings made of tokens, where the likelihood of each token is conditioned on all the tokens that come before it in the string. Formally, the likelihood of a string $w_0 w_1 \dots w_{n-1}$ is

$$p(w_{0:n-1}) = p(w_0) \prod_{t=1}^{n-1} p(w_t | w_{0:t-1}), \quad w_i \in \mathcal{A}, n \in \mathbb{N}. \quad (\text{B1})$$

The most common usage of LLM is to complete a partial string $w_0 w_1 \dots w_{q-1}$, or a “prompt”, into a complete string $w_0 w_1 \dots w_{n-1}$, by maximizing the likelihood of the latter:

$$p(w_{0:n-1}) \propto p(w_{q:n-1} | w_{0:q-1}) = \prod_{t=q}^{n-1} p(w_t | w_{0:t-1}), \quad w_i \in \mathcal{A}, q, n \in \mathbb{N}, q < n. \quad (\text{B2})$$

As the equation suggests, such completion occurs iteratively, one token at a time. In practice, the iteration stops after a fixed number of iterations, or when a special stopping token is generated.

State-of-the-art LLMs [66, 67, 7, 52] are generally implemented using the Transformer architecture, which is a multi-layer feedforward network equipped with attention mechanism [71]. The “large” in LLM describes both the scale of the models, which is usually billions of parameters, and the size of training corpora, which is usually trillions of tokens [67, 52]. An LLM is trained with

the objective that the output string $w_{1:n-1}$ resembles the real-world corpora and is coherent and reasonable. Common training procedures include (1) unsupervised pretraining where LLMs are trained to complete strings from general corpus auto-regressively, (2) supervised finetuning where the pretrained LLMs are finetuned on training samples for specific tasks, and (3) reinforcement learning where human feedback is put as training signals for societal value alignment.

C Empirical Study: Can LLMs List Them All?

Consider a task intellectually simple for human beings: list all strings of a fixed length using a fixed alphabet. This task is listed in Table 3, which stated that polynomial-time LLMs will hallucinate on this task. In this section, we empirically validate this statement.

The Task An LLM is required to list all the strings with length m using an alphabet $\mathcal{A}$. We denote this task as $L(m, \mathcal{A})$. For example, a solution to $L(2, \{a, b\})$ is “bb, ba, ab, bb”. For each $L(m, \mathcal{A})$, we run the LLM three times using three random seeds. The LLM is deemed successful in solving $L(m, \mathcal{A})$ if, in any of the three runs, the LLM’s output contains all and only strings of length m using alphabet $\mathcal{A}$. Note that duplicated entries are allowed in the output, for example, “aa, ab, ba, bb, ab” is a valid solution to $L(2, \{a, b\})$.

Models In the experiment we study two LLM families, namely Llama 2 [67], Llama 3 [46], and Generative Pretrained Transformers (GPT) [7, 52]. Models in both families have context windows of at least 4096 tokens. For Llama 2, we use its 70-billion-parameter version llama2-70b-chat-hf publicly accessible on HuggingFace [29]. For Llama 3, we use its 70-billion-parameter four-bit quantized version llama3-70B-Instruct-bnb-4bit [30]. For GPT, we use the 175-billion-parameter gpt-3.5-turbo-16k for GPT-3.5, gpt-4-0613 for the GPT-4 family with more parameters (GPT-4’s parameter number is not disclosed in the report [52]), and the latest GPT-4 variant gpt-4-turbo-2024-04-09.

Table C2: Evaluation results of LLMs on $L(m, \{a, b\})$ and $L(m, \{a, b, c\})$ tasks. A check mark $\checkmark$ indicates that the LLM successfully solved the problem and a cross mark $\times$ indicates the opposite. The exact parameter number for GPT-4 is not disclosed. While gpt-4-turbo-2024-04-09 was successful in $L(5, \{a, b, c\})$, it failed an equivalent task $L(5, \{x, y, z\})$, hence the asterisk $\checkmark^*$.

LLM	# Param	# Context (tokens)	$L(m, \{a, b\})$							$L(m, \{a, b, c\})$				
			$m \rightarrow$							1	2	3	4	5
llama2-70B-chat-hf	7.00×10^{10}	4,096	$\checkmark$	$\checkmark$	$\times$	$\times$	$\times$	$\times$	$\times$	$\checkmark$	$\times$	$\times$	$\times$	$\times$
llama3-70B-Instruct-bnb-4bit	7.00×10^{10}	8,000	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\times$	$\times$	$\times$	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\times$
gpt-3.5-turbo-16k	1.75×10^{11}	16,385	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\times$	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\times$
gpt-4-0613	$\geq 1.75 \times 10^{11}$	8,192	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\times$
gpt-4-turbo-2024-04-09	$\geq 1.75 \times 10^{11}$	128,000	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark^*$

Prompt We first prompt LLMs to complete a dialogue made of questions and answers. The following is used as the base prompt for all the empirical studies in this paper. It is adapted from [28] with modifications made to prompt the LLMs to answer directly instead of providing high-level descriptions or code snippets.

Base Prompt

You are a helpful, respectful and honest assistant. Always answer as helpfully as possible, while being safe. Your answers should not include any harmful, unethical, racist, sexist, toxic, dangerous, or illegal content. Please ensure that your responses are socially unbiased and positive in nature.

If you don’t know the answer to a question, please don’t share false information. However, if you know the answer, you should always share it in every detail and as requested. Always answer directly. Do not respond with a script or any approximation.

Then, we describe the task to the LLM, for example, $L(2, \{a, b\})$ is described as:

Task Description for $L(2, \{a, b\})$

List ALL the strings with a length of two characters and only contains characters "a" and "b". Do not miss a single string.

Result We feed the base prompt and the task descriptions for $L(m, \{a, b\})$ and $L(m, \{a, b, c\})$ with $m \in \mathbb{N}$ to the LLMs. The evaluation result is shown in Table C2. Consistent with claims in Table 3, all LLMs failed eventually as m grows. Surprisingly, LLMs fail for tasks with short answer, compared to the length of their context window. For example, the answer of $L(7, \{a, b\})$ is a list of $2^7 = 128$ strings, each with 7 characters. This results in an 896-character-long answer, which is much shorter than the context window of tested LLMs. Furthermore, the number of parameters and context size does not significantly impact LLMs’ performance on this task: they are equally poor.

D LLM Cannot Learn All Computable Linear Orders

As a supplement to Theorem 3, this section presents a concrete example of a computable function representing a linear order between strings. A linear order is a binary relation between strings, where for each pair of distinct strings (s_i, s_j) , either $s_i < s_j$ or $s_j < s_i$. The ordering relation is transitive and irreflexive. This relation is an abstraction of many real-world relations such as ranking of movies or songs, chronological ordering of events, alphabetical sorting of words, and so on. We further provide an empirical study following the theoretical results.

We consider only linear orders which are isomorphic to $\mathbb{N} = 0, 1, 2, \dots$ or $\mathbb{Z} = \dots, -3, -2, -1, 0, 1, 2, \dots$. Since the order is assumed to be computable, there is a ground truth function $f_{<}$ which determines if $s_i < s_j$ for any s_i and s_j . We define $f_{<}(s_i < s_j?) = \text{“yes”}$ if $s_i < s_j$ and “no” otherwise.

For ease of presentation, in this section, we abuse the notation of $h^{[n]}$ to denote the state of LLM h trained on $\{(“s_i < s_j?”), f_{<}(s_i < s_j?) \mid i, j < 2n\}$.

D.1 Theoretical Results

We present the following theorem that LLM cannot learn all computable linear orders, by adapting results from [20]. The theorem has been presented as Theorem 4 in Section 3, we restate it below for the ease of presentation:

Theorem. For all computable LLM h , there exists a computable ordering $<$ such that LLM h hallucinates when answering question “ $s_{2n+1} < s_{2n}?$ ” after being trained on training samples $\{(“s_i < s_j?”), f_{<}(s_i < s_j?) \mid i, j < 2n\}$, where $n \in \mathbb{N}$, $f_{<}(s_i < s_j?) = \text{“yes”}$ if $s_i < s_j$ and “no” otherwise.

Proof. We now construct a computable ordering $<$ where $h^{[n]}$ hallucinates for all n . Such ordering is constructed in stages, where in stage n , we define $<$ on all pairs s_i, s_j with $i, j < 2n + 2$.

Let $\mathcal{T}_n$ be the training samples $\{(“s_i < s_j?”), f_{<}(s_i < s_j?) \mid i, j < 2n\}$. Increasing n is equivalent to providing more training samples to the model. Let $h^{[n]}$ denote the LLM that has been trained on $\mathcal{T}_n$.

1. Start from stage 0.
2. By induction, by stage $n - 1$, relation $<$ has been defined between s_i, s_j with $i, j < 2n$. So in stage n , we only need to define the ordering between s_{2n}, s_{2n+1} , and $s_i, i < 2n$, as below:
 - (a) If $h^{[n]}(s_{2n+1} < s_{2n}?) = \text{“yes”}$, then
Define $s_i < s_{2n} < s_{2n+1}$, for $i < 2n$.
As a result, $f_{<}(s_{2n+1} < s_{2n}?) = \text{“no”} \neq h^{[n]}(s_{2n+1} < s_{2n}?)$.
 - (b) If $h^{[n]}(s_{2n+1} < s_{2n}?) = \text{“no”}$, then
Define $s_i < s_{2n+1} < s_{2n}$, for $i < 2n$.

As a result, $f_{<}(s_{2n+1} < s_{2n}?) = \text{"yes"} \neq h^{[n]}(s_{2n+1} < s_{2n}?)$.

3. Go to stage $n + 1$.

Then:

1. $<$ is computable because: (1) it is defined for all pairs of (s_i, s_j) and is defined using computation results from a computable LLM h , and (2) it is defined for all pairs (s_i, s_j) with $i, j \leq 2n + 1$ either at or before stage n .
2. For all n , $h^{[n]}(s_{2n+1} < s_{2n}?) \neq f_{<}(s_{2n+1} < s_{2n}?)$ because we explicitly defined $<$ to contradict $h^{[n]}$ in step 2a and 2b for all n .

□

As a result, an LLM will inevitably hallucinate when determining linear orders between strings in the controlled world. This result holds for any n , where the examples given to h is the ordering between any pair if (s_i, s_j) , with $i, j < 2n$.

Furthermore, it turns out that LLMs will make mistakes infinitely often when answering if a linear order is isomorphic to $\mathbb{N}$ or $\mathbb{Z}$, as shown in [4] for general learning algorithms:

Theorem D1. For all computable LLM h , there exists a computable ordering $<$ such that h makes errors when answering question “is ordering $<$ isomorphic to $\mathbb{N}$ or $\mathbb{Z}$?” after being trained on $\{(“s_i < s_j?”), f_{<}(s_i < s_j?) \mid i, j < 2n\}$, for infinitely many n , where $n \in \mathbb{N}$, $f_{<}(s_i < s_j?) = \text{"yes"}$ if $s_i < s_j$ and “no” otherwise.

Proof. We now construct a computable ordering $<$ where for infinitely many n , $h^{[n]}$ gives the wrong answer. Such ordering is constructed in stages, where in stage n , we define $<$ on all pairs s_i, s_j with $i, j < 2n + 2$.

1. Start from stage 0.
2. By induction, in stage $n - 1$, relation $<$ has been defined between s_i, s_j with $i, j < 2n$. So in stage n , we only need to define the ordering between s_{2n}, s_{2n+1} , and $s_i, i < 2n$, as below:
 - (a) If $h^{[n]}(\text{"N or Z?"}) = \mathbb{N}$, then
Define $s_{2n+1} < s_i < s_{2n}$, for $i < 2n$.
 - (b) If $h^{[n]}(\text{"N or Z?"}) = \mathbb{Z}$, then
Define $s_i < s_{2n} < s_{2n+1}$, for $i < 2n$.
3. Go to stage $n + 1$.

Then:

1. Relation $<$ is computable: (1) it is defined for all pairs of (s_i, s_j) and is defined using computation results from a computable LLM h , and (2) it is defined for all pairs (s_i, s_j) with $i, j \leq 2n + 1$ either at or before stage n .
2. The ordering given by $<$ is either isomorphic to $\mathbb{N}$ or $\mathbb{Z}$ because in every stage, we always add element above all elements in the finite order constructed in the previous stages, and sometimes below all these elements.
3. For infinite number of n , $h^{[n]}$ will answer the question wrongly, because
 - (a) If $h^{[n]}$ gives the answer $\mathbb{N}$ for infinitely many n , then step 2a is taken infinitely often, and the linear order is isomorphic to $\mathbb{Z}$.
 - (b) If $h^{[n]}$ gives the answer $\mathbb{Z}$ for almost all n , then step 2b is taken almost always, and the linear order is isomorphic to $\mathbb{N}$.

Thus, h gives wrong answers infinitely often. □

Theorem 4 and D1 show that computable LLMs will hallucinate on questions about both local and global properties of linear ordering $<$, no matter how many training samples are provided.

D.2 Empirical Study

In this section, we explore the capabilities of LLMs to discern linear order relations between binary integers. To prevent the LLMs from reciting the corpus and really “reason” using the provided information, we use an intentionally perplexing scheme. Specifically, symbol “\$” is used to represent the relation “less than”, or “ $<$ ” as generally written in corpus. Furthermore, character “b” is used to represent “0”, and “a” for “1”. A linear order “ $0 < 1$ ” in natural language thus translates to “b\$a” in our abstract representation.

The Task An LLM is provided with examples describing orders between binary numbers from zero (i.e., binary number 0_2 represented by b) up to m (e.g., binary number 10000_2 represented by abbbb). It is then asked if a statement “ xy$ ” is true, where x and y are strings representing binary numbers. This task is denoted $\omega(m)$.

There are two cases in $\omega(m)$: (1) both x and y are present in the samples, but their relation is not, and (2) at least one of x and y is not in the samples. We denote these two cases as $\omega_1(m)$ and $\omega_2(m)$, respectively. For each case, we randomly generate five pairs of strings for test. For each pair (s_i, s_j) , we generate two statements “ s_is_j$ ” and “ s_js_i$ ”. This results in ten test statements for each case.

We run LLM on these two cases using three random seeds. The LLM is deemed successful in solving $\omega_*(m)$ if, in any of the three runs, it correctly determine if each of the test statements is true. Furthermore, for $\omega_2(m)$, the LLM is also deemed successful if its answer is “unknown” for all the statements.

Models We use the same LLMs and configurations as in Appendix C.

Prompt The prompt begins with the formulation of the strings and the introduction of a partial order relation “\$” among them, as follows:

Definition of relation

There is a relation “\$” between strings made of characters “a” and “b”.

Given such strings x , y , and z , the relation has the following properties:

- (a) if xy$ is true, then yx$ is false,
- (b) if both xy$ and yz$ are true, then xz$ is true, and
- (c) xx$ is always false, for any x .

Following the properties, a list of samples satisfying the relation is provided.

Examples for $\Omega(m)$

Below are examples of strings that satisfy the relation “\$”.

Note that many other strings also satisfy the relation.

b\$a

a\$ab

ab\$aa

/* omitted for conciseness, the examples cover binary integers from 0 to m */

...

(omitted)

We finally ask the question using the following:

Task Description

Given the above information, determine if {string 1}\$ {string 2} is true.

If it is true, your answer must be “true”.

If it is false, your answer must be “false”.

If you do not know if it is true or false, you answer must be “unknown”.

Table D3: Evaluation results of LLMs on $\omega_*(m)$ tasks. A check mark $\checkmark$ indicates that the LLM successfully solved the problem and a cross mark $\times$ indicates the opposite. The exact parameter number for GPT-4 is not revealed.

LLM	# Param	# Context (tokens)	$\omega(1000_2)$		$\omega(10000_2)$	
			Cases $\rightarrow$	$\omega_1(1000_2)$	$\omega_2(1000_2)$	$\omega_1(10000_2)$
llama2-70B-chat-hf	7.00×10^{10}	4,096	$\times$	$\times$	$\times$	$\times$
llama3-70B-Instruct-bnb-4bit	7.00×10^{10}	8,000	$\times$	$\times$	$\times$	$\times$
gpt-3.5-turbo-16k	1.75×10^{11}	16,385	$\times$	$\times$	$\times$	$\times$
gpt-4-0613	$\geq 1.75 \times 10^{11}$	8,192	$\times$	$\times$	$\times$	$\times$
gpt-4-turbo-2024-04-09	$\geq 1.75 \times 10^{11}$	128,000	$\times$	$\times$	$\times$	$\times$

Result The results are shown in Table D3. Shockingly but expectedly, all LLMs failed in this task. For ω_1 tasks, examination of LLMs’ responses reveal that they are unable to use the transitive rule to deduce relations. For ω_2 tasks, LLMs tend to give inconsistent answers for “ $x\$y$ ” and “ $y\$x$ ”, for example, giving “unknown” for one but “true” for the other. This suggests a deficiency in their reasoning abilities.

E Identifying Limits on LLMs’ Capabilities

One may naturally think the other way round: what are the functions on which an LLM can possibly be hallucination-free? We give the following theorem as an “upper bound” of the capability of LLMs. We show that the set of functions on which LLM is hallucination-free is contained in a computably enumerable set of total computable functions, by adapting results and proofs from [3] and [20]:

Theorem E2. Let set $\mathcal{F}$ denote the set of all the ground truth functions w.r.t. which a computable LLM h can be hallucination-free after being trained on some finite number of training samples. Then $\mathcal{F}$ is contained in a computably enumerable set of total computable functions.

Proof. LLMs are trained on training samples. Therefore, to study what LLMs can reproduce, we need to first study all the possible training sets. We enumerate training sets of one sample, two samples, three samples, and so on. For example, training sets with one sample can be enumerated as

$$(\{(s_0, y_0)\}, \{(s_1, y_1)\}, \dots).$$

Training sets with two samples can be enumerated following the Cantor pairing function which maps a pair of natural number to a natural number:

$$(\{(s_0, y_0), (s_1, y_1)\}, \{(s_0, y_0), (s_2, y_2)\}, \{(s_1, y_1), (s_2, y_2)\}, \{(s_0, y_0), (s_3, y_3)\}, \dots).$$

We can then continue to list training sets of $n = 3, 4, \dots$ samples to get all the possible training sets.

Let $\mathcal{T}_n^i$ denote the i^{th} training set that contains n samples. We can list samples in it as $\mathcal{T}_n^i = \{(s_{i,0}, y_{i,0}), (s_{i,1}, y_{i,1}), \dots, (s_{i,n-1}, y_{i,n-1})\}$, where $n > 0$ and $y_{i,*}$ is the expected output for $s_{i,*}$ under some ground truth function. For example, $\mathcal{T}_1^0 = \{(s_0, y_0)\}$ and $\mathcal{T}_2^1 = \{(s_0, y_0), (s_2, y_2)\}$ according to our enumeration above.

Let h_n^i be an LLM trained on $\mathcal{T}_n^i$. For each $\mathcal{T}_n^i$, define a function f_n^i as follows:

$$\forall s \in \mathcal{S}, f_n^i(s) = \begin{cases} y_{i,j} & \text{if } s = s_{i,j} \text{ for some } j < n, \\ h_n^i(s) & \text{otherwise.} \end{cases} \quad (\text{E3})$$

Let $\mathcal{F}' = \{f_n^i \mid i, n \in \mathbb{N}\}$. We show (a) $\mathcal{F} \subseteq \mathcal{F}'$ followed by (b) $\mathcal{F}'$ is computably enumerable.

(a) First, we show that $\mathcal{F} \subseteq \mathcal{F}'$.

- Consider any $f \in \mathcal{F}$. According to the definition of $\mathcal{F}$, there exists $i, n \in \mathbb{N}$ such that h_n^i is hallucination-free w.r.t. f , so $\forall s \in \mathcal{S}, h_n^i(s) = f(s)$.
- h_n^i is trained on $\mathcal{T}_n^i$, so $h_n^i(s) = y_{i,j}$ if $s = s_{i,j}$ for some $j < n$. Therefore, by the definition of f_n^i , $\forall s \in \mathcal{S}, f_n^i(s) = h_n^i(s)$.

Table E4: Evaluation result for the task of $R(m, n)$, i.e., returning the n^{th} character of the input string of m characters. A check mark $\checkmark$ indicates that the LLM successfully solved the problem and a cross mark $\times$ indicates the opposite. The exact parameter number for GPT-4 is not revealed.

LLM	# Context (tokens) $m \rightarrow$	$R(m, 1)$				$R(m, 2)$				$R(m, 5)$			
		16	64	256	1024	16	64	256	1024	16	64	256	1024
llama2-70B-chat-hf	4,096	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$
llama3-70B-Instruct-bnb-4bit	8,000	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\times$
gpt-3.5-turbo-16k	16,385	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\times$	$\times$	$\times$
gpt-4-0613	8,192	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\times$	$\times$	$\times$
gpt-4-turbo-2024-04-09	128,000	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\times$

- Therefore, $\forall s \in \mathcal{S}, f_n^i(s) = f(s)$. This means that such ground truth f is f_n^i .

Repeating this reasoning for all $i, n \in \mathbb{N}$, we find that any $f \in \mathcal{F}$ must be some $f_n^i \in \mathcal{F}'$. Therefore, $\mathcal{F} \subseteq \mathcal{F}'$.

(b) Then, we show that $\mathcal{F}'$ is a computably enumerable set of total computable functions. First, every f_n^i is total computable because LLM h_n^i is total computable. Second, set $\{\mathcal{T}_n^i \mid i, n \in \mathbb{N}\}$ is computably enumerable and we have defined each f_n^i for a $\mathcal{T}_n^i$, so there is an one-to-one mapping between each $\mathcal{T}_n^i$ and f_n^i , which means $\mathcal{F}'$ is computably enumerable. As a result, $\mathcal{F}'$ is a computably enumerable set of total computable functions.

Therefore, $\mathcal{F}$ is contained in a computably enumerable set of total computable functions. $\square$

Theorem E2 offers another view to our negative answer to the fundamental question: there are many real-world problems whose answers are not computable at all, let alone being contained in a computably enumerable set of total computable functions, and therefore cannot be solved by LLMs. For example, the problem of deciding if any program will halt is not computable.

In defence for LLMs, we present the following theorem that some ground truth functions can be learned by some LLMs, also by adapting results from [3] and [20]:

Theorem E3. For all computably enumerable sets $\mathcal{C}$ of total computable functions, there exist an LLM h such that for all computable functions $f \in \mathcal{C}$, there exists a training set of up to k samples $\{s_0, s_1, \dots, s_k\}$, such that $h^{[k]}$ is hallucination free on f .

Theorem E3 implies that there are some real-world problems where LLMs can be trained to be hallucination-free, as long as their ground truth functions are contained in a computably enumerable set of total computable functions.

For example, the task of returning the n^{th} character of input string of length $m \in \mathbb{N}$ is learnable by some (and not all) LLMs. Denote this task as $R(m, n)$. We consider the following cases $\{R(m, n) \mid m \in \{16, 64, 256, 1024\}, n \in \{1, 2, 5\}\}$. For every n , we provide LLMs discussed in Appendix C with five examples showing the n^{th} character of some randomly generated 16-character strings. The LLMs are then tested on a different set of five randomly generated strings. Each LLM is tested three times using different random seeds. An LLM is considered successful in a case if, in any of the three runs, it correctly returns the n^{th} character for all five test strings in that case. As shown in Table E4, all LLMs performed well in $R(*, 1)$. However, Llama 2 fails $R(*, 2)$ and all of the LLMs fails $R(m, 5)$ for $m \geq 256$. The result is surprising and implies that (1) training corpora have not covered enough training samples for counting and indexing and/or (2) state-of-the-art LLMs have not achieved the theoretical limits of their abilities. This calls for further research into training techniques and architecture design for LLMs.

F In Defence of LLMs and Hallucination

Although we have shown that LLMs will inevitably hallucinate, this does not undermine their tremendous value in enhancing productivity. Moreover, hallucination itself should not be viewed entirely negatively. In this section, we would like to defend the usage of LLMs and their hallucinatory inclinations.

First, the decision to use an LLM is fundamentally a trade-off between precision and efficiency, which is largely determined by their application. LLMs excel in processing and structuring information at scales and speeds unachievable by humans, facilitating rapid decision-making and idea generation. In scenarios where speed and volume of information processing are overwhelming, the occasional inaccuracies of LLMs are acceptable compromises. Conversely, in situations where precision is critical, the outputs of LLMs can (and must) be verified and supplemented with human supervision. It is notable that while LLMs cannot learn *all* computable ground truth functions f , it can learn *some* classes of f (see Appendix E) and can be useful therein. The key is not to view LLMs as infallible sources of truth but as powerful assistive tools for information retrieval, analysis, summarisation, and presentation.

Moreover, it is crucial to recognize that LLMs are continuously evolving. Such evolution includes advancements in model architecture, data processing techniques, training methodologies, and error correction strategies. Over time, we anticipate that the nature of hallucination will be better understood by human researchers and users. While it is impossible to completely eliminate hallucination, we are optimistic that its severity may be controlled and reduced for many applications.

Finally, hallucination is not completely detrimental. In art, literature, and design, the unintentional, unpredictable, or even nonsensical outputs from LLMs could inspire human creators. Such deviation from facts can lead to unique perspectives that a strictly factual and precise system might never generate. In this sense, the hallucinatory aspect of LLMs should be regarded positively as a source of inspiration, innovation, and creativity.

G Comparison with PAC and Online Learnability

This section briefly discusses the difference between the analysis in this paper and learnability in PAC and online learning theories. Specifically, we explain why PAC and online unlearnability do not answer Question 1.

G.1 PAC Unlearnability does not answer Question 1

Our formal world is defined by computable functions. The set of all computable functions is not PAC learnable. This translates to the following statement:

Statement 1. There exists no $O\left(\text{Polynomial}\left(\frac{1}{\epsilon}, \frac{1}{\delta}\right)\right)$ -time algorithm that can find an LLM that has ϵ or lower hallucination rate in all formal worlds with probability $(1 - \delta)$, for all distributions over input strings, and for all $0 < \epsilon, \delta < \frac{1}{2}$.

Theorem 1-2 in this paper translate to the following statement:

Statement 2. There is no computably enumerable sets of LLMs that has no hallucination in all formal worlds.

Statement 1 and Statement 2 are different. Statement 1 describes practical learnability within certain time complexity and error rate, while Statement 2 describes learnability regardless of those assumptions. Therefore, Statement 2 is more relevant to Question 1.

G.2 Online Unlearnability does not answer Question 1

In this paper, hallucination is not directly linked to unbounded mistakes, and thus differs from the conventional definition of being not online learnable. An LLM can be hallucination-free while making an unbounded (but finite) number of mistakes.

We show that there exists a class $\mathcal{F}$ that has infinite Littlestone dimension [41], but there exists an LLM, when trained using Procedure 1, will not hallucinate on it. Consider $\mathcal{F} : \{f : f(s) = 0 \text{ for almost every } s, s \in \mathcal{S}\}$. In other words, $\forall f \in \mathcal{F}, f(s) \neq 0$ for an arbitrary but finite number of $s \in \mathcal{S}$. $\mathcal{F}$ has an infinite Littlestone dimension, thus resulting in unbounded mistakes in the online learning setting. However, an LLM as used in this paper can learn this class by predicting the zero-extension of the input seen so far in the training. There exists an state of this LLM which would not make any error beyond the point when all non-zero points for f has been provided in the training. In this case, this LLM does not hallucinate.

Therefore, conventional notions of online unlearnability does not answer Question 1.

H Open Problems

Finally, we list some challenging and fundamental open problems stemming from our discussion:

- What real-world problems can be provably addressed by LLMs without hallucination? The problem can be further divided: (1) what is the theoretical computational capability of an LLM, (2) what are the complexities of real-world problems, and (3) how can LLMs achieve their theoretical capabilities so that hallucination can be eliminated in limited realms?
- How can hallucination be detected and corrected for real-world problems using external knowledge sources and reasoning tools? Can LLMs find the appropriate external tools for solving real-world problems? Should we train an LLM that does everything or an LLM that can use proper tools for specific problems?
- How to quantify the risk of hallucination for real-world problems? Quantifying hallucination risk is an important step to identify on which problems can LLMs be safely deployed, even with inevitable hallucination. Probabilistic methods could be helpful in bounding such risk.

I Notes on Empirical Study

All the empirical studies presented in this paper were conducted between December 2023 and May 2024. Inference of Llama 2 was conducted using four NVIDIA A100 GPUs. Inference of four-bit quantized version of Llama 3 was conducted using one NVIDIA A100 GPU.